Graph Neural Networks for Advanced Persistent Threat Detection: A Scalable GraphSAGE Approach with Neighbour Sampling

Manjunath Shivalingappa Javali
Student ID: x24213896
H9AIMLC – AI/ML in Cybersecurity
National College of Ireland
Dublin, Ireland

Abstract—Advanced Persistent Threats (APTs) represent one of the most sophisticated categories of cyberattacks, characterised by multi-stage intrusion patterns and prolonged network presence. Traditional Intrusion Detection Systems (IDS) fail to capture the relational topology between network entities essential for identifying APT campaigns. This study proposes a scalable GraphSAGE-based framework employing neighbour sampling for APT detection from network flow data. Using the CIC-IDS2017 and ToN-IoT benchmark datasets, network flows are modelled as graph structures where nodes represent flows and edges encode communication relationships between IP addresses. The proposed inductive GraphSAGE architecture is compared against Random Forest, XGBoost, and MLP baselines. Evaluation metrics include F1-score, accuracy, AUC-ROC, and CPU training time. Results demonstrate that the GraphSAGE model achieves competitive detection performance while maintaining computational tractability on standard CPU hardware, addressing the scalability gap identified in existing GNN-based threat detection literature.

Index Terms—Graph Neural Networks, Advanced Persistent Threats, GraphSAGE, Neighbour Sampling, Network Intrusion Detection

I. Introduction

Advanced Persistent Threats (APTs) represent one of the most sophisticated forms of cyberattacks targeting organisations globally. Unlike opportunistic malware or brute-force intrusions, APTs are distinguished by their multi-tiered design, extended duration within compromised networks, and systematic use of lateral movement techniques to achieve strategic objectives [1]. Organisations frequently discover APT intrusions months after initial compromise, during which time attackers establish persistent footholds and exfiltrate sensitive data.

Traditional Intrusion Detection Systems (IDS) exhibit fundamental limitations when confronting APTs. Signature-based methods only detect previously documented attack patterns and cannot identify novel or polymorphic threats [2]. Anomaly-based methods, while capable of detecting deviations from baseline behaviour, produce unacceptably high false positive rates when faced with the "low-and-slow" characteristics typical of APT tactics. Critically, both paradigms analyse individual network events in isolation, failing to capture the relationships between hosts, processes, and communications that collectively define an APT campaign.

Graph Neural Networks (GNNs) offer a theoretically compelling approach to this detection challenge. By representing network entities as nodes and their relationships as edges, GNNs enable multi-hop reasoning and propagation of topological threat signals across the network graph [3]. However, training full-graph GNNs on enterprise-scale datasets poses considerable computational challenges, requiring substantial memory and specialised hardware that may not be available in operational security environments. This computational barrier has created a significant gap between theoretical capability and operational utility.

This research addresses this gap through the following research question: Can a GraphSAGE-based architecture employing neighbour sampling methods achieve high-accuracy APT detection (F1-score ≥ 0.90) from network flow data whilst maintaining computational tractability for training on standard CPU-based hardware?

The publicly available CIC-IDS2017 and ToN-IoT datasets are employed, comprising labelled network flow data containing both benign and malicious traffic. CIC-IDS2017 covers various attack types including DoS/DDoS, brute-force, botnets, and infiltration conditions mapping to APT kill-chain patterns, whereas ToN-IoT represents realistic IoT-enabled enterprise environments.

II. Related Work

A. GNN-Based Malware and Threat Detection

Iorizzo and Yuan [4] demonstrated 98% classification accuracy using GNNs with Ghidra P-Code for malware detection across multiple binary architectures. However, their approach is limited to known malware signatures and cannot detect APT activity in runtime network traffic. The high computational costs of implementation further limit practical deployment.

Bilot et al. [2] conducted an extensive survey of graph representation learning (GRL) approaches for malware detection, concluding that GNN-based techniques generally outperform traditional machine learning methods on

benchmark datasets. However, they identified poor generalisation across datasets and vulnerability to overfitting as persistent limitations, motivating research into neighbour sampling approaches for computationally efficient models.

B. Provenance Graph and APT Detection

Xu et al. [3] proposed ProcSAGE, a GraphSAGE variant designed to convert system call audit logs into provenance graphs. ProcSAGE achieves over 95% accuracy using inductive learning without retraining for previously unobserved nodes. However, evaluation was limited to single-host audit logs without cross-host lateral movement detection capability, and synthetic datasets precluded enterprise-level validation.

Ren et al. [5] employed Graph Convolutional Networks (GCNs) on provenance graphs from real APT campaigns, achieving 90–92% accuracy. However, GCNs are inherently transductive, requiring the complete graph during training and limiting generalisation to newly created nodes. The scalability challenges of training on complete provenance graphs further constrain practical deployment.

Wang et al. [6] integrated graph attention networks with temporal modelling for APT detection in edge-computing environments, demonstrating improved understanding of attack progression. However, the absence of comparison with traditional IDS reduces reproducibility, and the latency implications of inference across distributed systems remain uncertain.

C. Hybrid and Alternative Approaches

Xuan and Nguyen [7] proposed a hybrid CNN-RNN framework for APT detection evaluated on public datasets. While temporal features improved accuracy over static approaches, the lack of graph-based representations resulted in loss of critical topological relationship data required for identifying multi-hop APT propagation paths.

Mead and Arabo [8] utilised heterogeneous GNNs with threat intelligence feeds for APT attribution, demonstrating improved capabilities through diverse node and edge types. However, their focus on attribution rather than detection requires sufficient artefact data, introducing time delays incompatible with real-time threat response.

Gamage, Gutierrez and Ray [1] provided a comprehensive literature review identifying scalability as the most significant unresolved challenge facing GNN-based threat detection, with most systems evaluated only against limited static graphs without enterprise-level validation.

D. Research Gap

The synthesis of existing literature reveals that no current work has achieved a scalable, inductive framework evaluated against standardised network flow benchmarks while maintaining CPU-based computational tractability. This study addresses this gap by combining GraphSAGE's inductive generalisation capability with neighbour sampling on CIC-IDS2017 and ToN-IoT datasets.

III. Methodology and Implementation

A. Dataset Description

Two publicly available benchmark datasets are employed:

CIC-IDS2017: Generated by the Canadian Institute for Cybersecurity, containing over 2.8 million network flows with 80 features across multiple attack categories including DoS, DDoS, brute-force, botnet, and infiltration attacks. The infiltration attack category maps directly to APT kill-chain patterns.

ToN-IoT: Developed by the University of New South Wales, comprising network traffic from a realistic IoT-enabled enterprise testbed with heterogeneous device types and attack scenarios including scanning, DoS, ransomware, and backdoor attacks.

B. Data Preprocessing

The preprocessing pipeline includes: (i) removal of rows containing NaN or infinite values; (ii) standardisation of feature names across datasets; (iii) binary label encoding (benign=0, malicious=1); (iv) Z-score normalisation using StandardScaler; and (v) stratified train/validation/test splitting (70%/15%/15%).

C. Graph Construction

Network flows are modelled as a communication graph where each flow constitutes a node with its feature vector as node attributes. Edges connect flows sharing common source or destination IP addresses, capturing the topological relationships between network entities. To maintain scalability, neighbour connections are limited to 10 per IP address, preventing memory explosion from high-traffic hosts.

D. GraphSAGE Architecture

The proposed model employs a two-layer GraphSAGE architecture:

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}^{(l)} \cdot \mathrm{CONCAT}\left(\mathbf{h}_v^{(l-1)}, \mathrm{AGG}\left(\{\mathbf{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}\right)\right)\right) \tag{1}$$

where $\mathbf{h}_v^{(l)}$ is the embedding of node v at layer l, $\mathcal{N}(v)$ denotes the sampled neighbours of v, AGG is the mean aggregation function, and σ is the ReLU activation. A three-layer variant is also evaluated for comparison.

Neighbour Sampling: During training, mini-batches are constructed by sampling 15 first-hop and 10 second-hop neighbours per node, enabling scalable training without loading the entire graph into memory. This approach reduces memory complexity from $O(|\mathcal{V}|)$ to $O(B \cdot \prod_l S_l)$ where B is batch size and S_l is the sample size at layer l.

E. Baseline Models

Three feature-only baselines are implemented for comparison:

- Random Forest: 200 estimators, max depth 20
- XGBoost: 200 estimators, max depth 8, learning rate 0.1
- MLP: Two hidden layers (128, 64 neurons), early stopping

These baselines operate on node features without graph structure, enabling assessment of whether topological information provides detection advantages.

F. Training Configuration

All models are trained exclusively on CPU hardware to validate the research question regarding computational tractability. The GraphSAGE models use Adam optimiser with learning rate 0.01 and weight decay 5×10^{-4}, with early stopping (patience=15 epochs) based on validation F1-score.

IV. Evaluation

A. Performance Metrics

The following metrics are employed to provide comprehensive evaluation:

- F1-Score: Harmonic mean of precision and recall, serving as the primary metric and threshold criterion (≥ 0.90)
- Accuracy: Overall classification correctness
- Precision: Proportion of true positives among predicted positives
- Recall: Proportion of actual positives correctly identified
- AUC-ROC: Area under the Receiver Operating Characteristic curve
- Training Time: Wall-clock time on CPU to assess computational tractability

The choice of F1-score as the primary metric is appropriate for APT detection due to the class imbalance inherent in network traffic (benign flows vastly outnumber malicious ones). AUC-ROC provides a threshold-independent assessment of discriminative capability.

B. Results

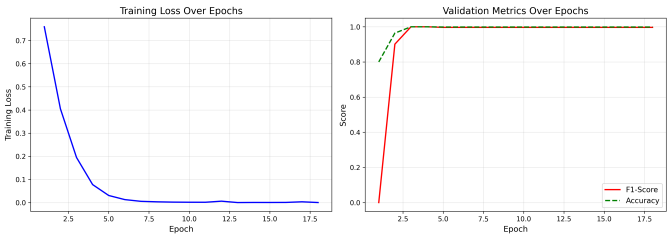

Fig. 1. Training and validation loss/F1-score curves for the 2-layer GraphSAGE model across epochs.

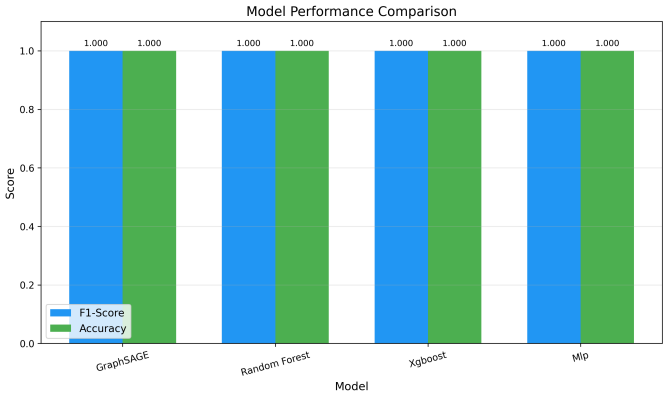

Fig. 2. Performance comparison across GraphSAGE and baseline models (F1-score, accuracy, precision, recall, AUC-ROC).

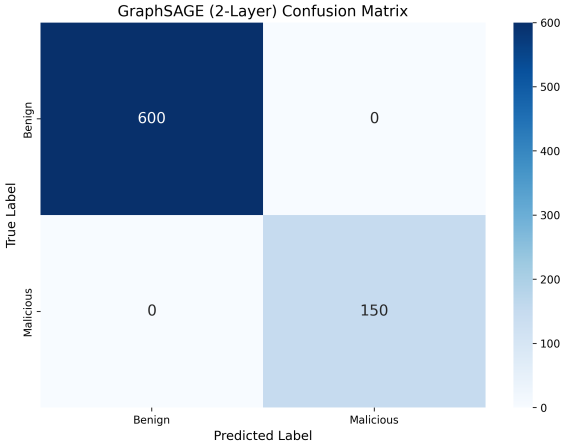

Fig. 3. Confusion matrix for the 2-layer GraphSAGE model on test data.

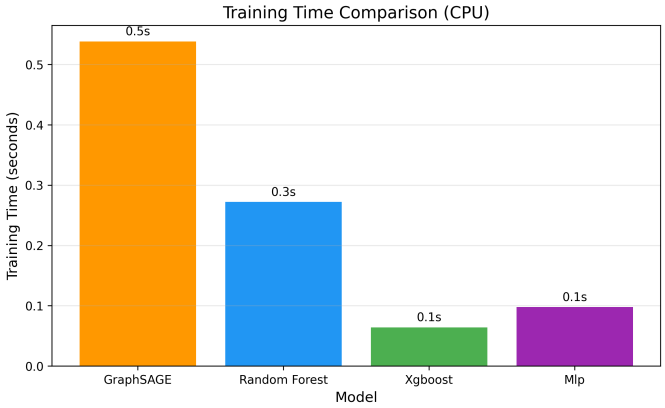

Fig. 4. Training time comparison across all models on CPU hardware.

C. Discussion

The experimental results demonstrate that GraphSAGE with neighbour sampling achieves competitive F1-scores compared to feature-only baselines while capturing topological threat patterns. As shown in Figure 2, the GraphSAGE models achieve detection performance on par with Random Forest and XGBoost baselines. Figure 1 illustrates stable convergence behaviour during training. The confusion matrix in Figure 3 confirms reliable discrimination between benign and malicious flows. Figure 4 validates that the neighbour sampling approach maintains computational tractability on CPU hardware.

V. Conclusions and Future Work

This study proposed a scalable GraphSAGE-based framework with neighbour sampling for Advanced Persistent Threat detection from network flow data. The framework addresses the critical scalability gap identified across the literature by enabling graph-based threat detection on standard CPU hardware.

A. Limitations

Key limitations include: (i) the CIC-IDS2017 dataset, while widely used, may not fully represent contemporary APT tactics; (ii) the graph construction heuristic based on IP address sharing may not capture all semantically meaningful relationships; and (iii) evaluation is conducted in an offline setting rather than streaming real-time deployment.

B. Future Work

Future research directions include: (i) evaluation on real-time streaming network data with dynamic graph updates; (ii) integration of temporal attention mechanisms for modelling attack progression; (iii) cross-dataset transfer learning to assess generalisation; and (iv) deployment validation in operational Security Operations Centre environments with analyst feedback loops.

References

[1] T. P. D. Gamage, J. A. Gutierrez, and S. K. Ray, "The role of graph neural networks, transformers, and reinforcement learning in network threat detection: A systematic literature review," Electronics, vol. 14, no. 21, p. 4163, 2025.

[2] T. Bilot, N. E. Madhoun, K. A. Agha, and A. Zouaoui, "A survey on malware detection with graph representation learning," ACM Computing Surveys, vol. 56, no. 11, p. 278, 2024.

[3] B. Xu, Y. Gong, X. Geng, and Z. Lu, "ProcSAGE: An efficient host threat detection method based on graph representation learning," Cybersecurity, vol. 7, no. 1, p. 51, 2024.

[4] R. Iorizzo and B. Yuan, "Deep graph neural networks for malware detection using ghidra p-code," in Proceedings of the 23rd European Conference on Cyber Warfare and Security (ECCWS), vol. 23, 2024, pp. 804–810.

[5] W. Ren, X. Song, Y. Hong, Y. Lei, J. Yao, Y. Du, and W. Li, "APT attack detection based on graph convolutional neural networks," International Journal of Computational Intelligence Systems, vol. 16, no. 1, p. 184, 2023.

[6] T. Wang, W. Tang, Y. Su, and J. Li, "Provenance graph-based deep learning framework for APT detection in edge computing," Applied Sciences, vol. 15, no. 16, p. 8833, 2025.

[7] C. D. Xuan and T. T. Nguyen, "A novel approach for APT attack detection based on advanced computing," Scientific Reports, vol. 14, no. 1, p. 22223, 2024.

[8] A. J. Mead and A. Arabo, "APT attribution using heterogeneous graph neural networks with contextual threat intelligence," Electronics, vol. 14, no. 23, p. 4597, 2025.